

HANDLERS & TEMPLATES

Reactive Task Execution and
Dynamic Configuration with Jinja2

Professional Reference Documentation

Version 1.0

Table of Contents

- 1. Introduction
- 2. Handlers
 - 2.1 What a Handler Is
 - 2.2 Defining and Notifying Handlers
 - 2.3 When Handlers Run
 - 2.4 Handler Ordering and Deduplication
 - 2.5 Listening to Multiple Events
 - 2.6 Forcing a Handler to Run
 - 2.7 Handlers in Roles
 - 2.8 Common Pitfalls
- 3. Templates with Jinja2
 - 3.1 What a Template Is
 - 3.2 The template Module
 - 3.3 Injecting Variables
 - 3.4 Filters
 - 3.5 Conditionals in Templates
 - 3.6 Loops in Templates
 - 3.7 Template Inheritance
 - 3.8 Whitespace Control
 - 3.9 Validating Templates
- 4. Handlers and Templates Together
- 5. Practical Examples
 - 5.1 nginx Virtual Host
 - 5.2 Application Configuration from Facts
 - 5.3 Multi-Host Configuration File
- 6. Best Practices
- 7. Quick Reference

1. Introduction

A well-written Ansible playbook does not restart services on every run, and it does not hard-code configuration values into template files. Two features make this possible:

- **Handlers** — tasks that run only when notified by another task that reported a change. They are the correct mechanism for service restarts, cache clearing, and similar reactive actions.
- **Templates** — files ending in `.j2` that are rendered by the Jinja2 engine and deployed by the `template` module. Templates inject variables, loops, and conditionals directly into configuration text.

Together, they produce idempotent playbooks that change only what needs to change and generate host-specific configuration from a single source of truth. This document covers both features in depth and closes with three practical examples that combine them.

2. Handlers

2.1 What a Handler Is

A handler is a task with a name. It looks like any other task, but it is declared under the `handlers` key of a play rather than under `tasks`, and it runs only when another task **notifies** it. Notification is triggered when the notifying task reports a change — that is, when the module actually modified the system. If the task found nothing to do, the handler is not notified.

This is the mechanism that keeps playbooks idempotent. A configuration file that is already correct produces no change, so no notification is sent, and the service is not restarted unnecessarily.

2.2 Defining and Notifying Handlers

Minimal example — restart nginx only when its config changes:

```
---
- name: Configure nginx
  hosts: webservers
  become: true

  tasks:
    - name: Deploy the nginx configuration
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
        notify: Restart nginx

  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

The `notify` value must match the handler name exactly, character for character. Notifications are case-sensitive.

2.3 When Handlers Run

Handlers run at specific points in the play, not immediately when notified. The rules are:

- **End of the play** — by default, all notified handlers run once, at the very end of the play, after all tasks have completed.
- **Once per play** — even if ten tasks notify the same handler, it runs exactly once.
- **Only if notified** — a handler that is never notified does not run.
- **In declaration order** — if multiple handlers are notified, they run in the order they are defined, not in the order they were notified.
- **Before the next play** — when a play finishes, its handlers are flushed before the next play begins.

Timeline for a play with two notifying tasks:

```
Task 1 (changed) -> notify handler
Task 2 (ok)       -> no notify
Task 3 (changed) -> notify same handler again
Task 4 (ok)
--- end of tasks ---
Handler runs once (deduplicated)
```

To force handlers to run before the end of the play, insert a meta: `flush_handlers` task.

```
- name: Install and configure nginx
  hosts: webservers
  become: true
  tasks:
    - name: Deploy configuration
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        notify: Restart nginx

    - name: Flush handlers now
      ansible.builtin.meta: flush_handlers

    - name: Verify the site is reachable
      ansible.builtin.uri:
        url: http://localhost/
        status_code: 200
      register: site_check
      until: site_check.status == 200
      retries: 5
      delay: 3
```

The `flush_handlers` call ensures `nginx` is restarted before the URI check runs, rather than after.

2.4 Handler Ordering and Deduplication

Handlers run in the order they are declared under the `handlers` key, regardless of notification order. This makes the sequence deterministic and easy to reason about.

```
handlers:
  - name: Reload systemd
    ansible.builtin.systemd_service:
      daemon_reload: true

  - name: Restart myapp
    ansible.builtin.service:
      name: myapp
      state: restarted

# Even if Restart myapp is notified first, Reload systemd runs first
# because it is declared first.
```

If you need a handler to run only after another handler has finished, chain them: have the first handler notify the second.

2.5 Listening to Multiple Events

A handler may declare a list of listen topics. Any task can then notify by topic name instead of by handler name. This decouples task definitions from handler names and allows one handler to respond to several events.

```
- name: Configure the application
  hosts: appservers
  become: true

tasks:
  - name: Update application configuration
    ansible.builtin.template:
      src: app.conf.j2
      dest: /etc/myapp/app.conf
      notify: reload application

  - name: Update TLS certificate
    ansible.builtin.copy:
      src: files/server.crt
      dest: /etc/myapp/server.crt
      notify: reload application

handlers:
  - name: Reload the application
    ansible.builtin.service:
      name: myapp
      state: reloaded
    listen:
      - reload application
      - "restart application"
```

2.6 Forcing a Handler to Run

Occasionally a handler must run regardless of whether anything changed — for example, on the first deploy or during a forced convergence. In that case, add a task that explicitly notifies it, or use `changed_when: true` on a task that should trigger the notification.

```
- name: Force a reload after a manual trigger
  ansible.builtin.debug:
    msg: "Forcing application reload"
  changed_when: true
  notify: Reload the application
```

2.7 Handlers in Roles

Handlers declared inside a role live in `roles/<role>/handlers/main.yml`. They are automatically available to any play that includes the role, and are notified the same way as play-level handlers.

```
roles/
|-- nginx/
|   |-- tasks/
|   |   |-- main.yml
|   |-- templates/
|   |   |-- nginx.conf.j2
|   |-- handlers/
|   |   |-- main.yml
```

Contents of `roles/nginx/handlers/main.yml`:

```
---
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted

- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

2.8 Common Pitfalls

- **Name mismatch.** A typo in the `notify` value silently fails — the handler is never called. Always copy the handler name exactly.
- **Expecting immediate execution.** Handlers run at the end of the play. Use `meta: flush_handlers` when a later task depends on the restart.
- **Notify on a task that never changes.** If the module is idempotent and the system is already in the desired state, no notification is sent and the handler does not run. That is the intended behaviour.
- **Not using handlers at all.** Putting `state: restarted` directly in a task causes a restart on every run and breaks idempotence.
- **Handlers not flushed before failure.** If a task fails, remaining handlers may still run at the end of the play, depending on configuration. Use `--force-handlers` to force them in failure scenarios.

3. Templates with Jinja2

3.1 What a Template Is

A template is a text file ending in `.j2` that contains placeholders written in the Jinja2 templating language. When Ansible renders a template, it substitutes each placeholder with the value of the referenced variable, evaluates any loops or conditionals, and produces the final output text. The rendered text is then copied to the managed node.

Common uses:

- Configuration files that differ per host or per group.
- Files that must include the current hostname, IP, or other fact.
- Files whose content depends on the environment (staging vs. production).
- Files that iterate over a list of backends, users, or virtual hosts.

3.2 The template Module

The `ansible.builtin.template` module renders a local `.j2` file and writes the result to the target. It accepts most of the same arguments as `copy` (owner, group, mode, backup, validate).

```
- name: Deploy the application configuration
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/myapp/app.conf
    owner: deploy
    group: deploy
    mode: '0640'
    backup: true
    notify: Restart myapp
```

By default the module searches for `src` in `templates/` relative to the playbook. Within a role, the search path is `roles/<role>/templates/`.

3.3 Injecting Variables

Any variable available to the play — play vars, host vars, group vars, or facts — is available inside the template. Reference it with the standard `{{ }}` syntax.

Template — `templates/app.conf.j2`:

```
# Application configuration
# Generated by Ansible -- do not edit by hand

[server]
host      = {{ ansible_facts.hostname }}
port      = {{ app_port }}
workers   = {{ app_workers | default(4) }}
log_level = {{ log_level | default('info') }}

[database]
host      = {{ db_host }}
port      = {{ db_port | default(5432) }}
name      = {{ db_name }}
user      = {{ db_user }}
password  = {{ db_password }}

[environment]
name      = {{ environment }}
region    = {{ region | default('us-east-1') }}
```

Rendering this template on `web01.example.com` with `app_port: 8080` produces a concrete configuration file. Running the same playbook on another host produces a different file, generated from the same source.

3.4 Filters

Filters transform values inside a template. They are applied with the pipe operator `|` and may be chained.

Filter	Purpose	Example
<code>default(x)</code>	Fallback when undefined.	<code>{{ port default(80) }}</code>
<code>upper / lower</code>	Change case.	<code>{{ env upper }}</code>
<code>length</code>	Length of a list or string.	<code>{{ servers length }}</code>
<code>join(sep)</code>	Join a list into a string.	<code>{{ dns join(', ') }}</code>
<code>basename / dirname</code>	Path components.	<code>{{ path basename }}</code>
<code>to_json / to_yaml</code>	Serialize a structure.	<code>{{ config to_json }}</code>
<code>bool</code>	Force a boolean value.	<code>{{ flag bool }}</code>
<code>quote</code>	Shell-quote a value.	<code>{{ value quote }}</code>
<code>int / float</code>	Cast to a number.	<code>{{ count int }}</code>
<code>regex_replace</code>	Regex substitution.	<code>{{ s regex_replace('a', 'b') }}</code>
<code>hash / password_hash</code>	Cryptographic hash.	<code>{{ pw password_hash('sha512') }}</code>
<code>selectattr / rejectattr</code>	Filter a list of dicts.	<code>{{ users selectattr('admin') list }}</code>

Table 1 — Frequently used Jinja2 filters.

3.5 Conditionals in Templates

Templates support conditional blocks with `{% if %}`, `{% elif %}`, and `{% else %}`. Note that statements use `{% %}`, while expression output uses `{{ }}`.

```

server {
    listen {{ app_port }};
    server_name {{ server_name }};

{% if ssl_enabled | default(false) %}
    listen 443 ssl;
    ssl_certificate      /etc/ssl/certs/{{ server_name }}.crt;
    ssl_certificate_key  /etc/ssl/private/{{ server_name }}.key;
{% endif %}

    root {{ document_root | default('/var/www/html') }};

{% if environment == 'production' %}
    access_log /var/log/nginx/{{ server_name }}.access.log;
    error_log  /var/log/nginx/{{ server_name }}.error.log;
{% else %}
    access_log off;
{% endif %}
}

```

The same conditional logic available in when expressions is available in template conditionals: comparisons, membership tests, `is defined`, and the `and` / `or` operators.

3.6 Loops in Templates

The `{% for %}` statement iterates over a list or dictionary. Inside the loop, a special loop object provides metadata such as `loop.index`, `loop.first`, and `loop.last`.

```

upstream app_backend {
{% for server in app_servers %}
    server {{ server.host }}:{{ server.port | default(8080) }} weight={{ server.weight | default(1) }};
{% endfor %}
}

{% for vhost in virtual_hosts %}
server {
    listen 80;
    server_name {{ vhost.name }};

    location / {
        proxy_pass http://app_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
{% if vhost.websocket | default(false) %}
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
{% endif %}
    }
}
{% endfor %}

```

Loops may be nested, but deeply nested templates become hard to read. When a configuration requires many levels, consider generating a list of dictionaries in a variable and iterating once.

3.7 Template Inheritance

Jinja2 supports template inheritance with `{% extends %}` and `{% block %}`. This lets you define a base template once and override specific sections in child templates.

Base template — `templates/base.conf.j2`:

```
# {{ app_name }} configuration
# Managed by Ansible

{% block settings %}
# Default settings (overridden by children)
debug = false
{% endblock %}

{% block database %}
[database]
host = {{ db_host }}
{% endblock %}
```

Child template — `templates/production.conf.j2`:

```
{% extends "base.conf.j2" %}

{% block settings %}
debug = false
workers = {{ app_workers }}
{% endblock %}
```

Note that the child may extend a template by name; the search path includes the `templates/` directory of the current playbook or role.

3.8 Whitespace Control

By default, Jinja2 preserves the whitespace surrounding block tags, which can leave unwanted blank lines in the rendered output. Two mechanisms control this.

- **Trim markers** — add a minus sign inside the tag: `{%- ... -%}` trims whitespace on the left and right respectively.
- **Global trimming** — set `trim_blocks` and `lstrip_blocks` in `ansible.cfg` or on the play to apply the rule everywhere.

```
# ansible.cfg
[defaults]
jinja2_extensions = jinja2.ext.do,jinja2.ext.loopcontrols
# Trimming is controlled at the template level with trim_blocks / lstrip_blocks
# passed via the template module, or globally via the environment.
```

Per-tag trimming inside a template:

```
{% for user in users -%}
{{ user.name }}:{{ user.uid }}
{% endfor -%}
```

The `-%` at the end of the `for` and `endfor` tags removes the newline after each, producing a dense list with no blank lines.

3.9 Validating Templates

A malformed configuration file can take a service offline. Ansible supports a `validate` parameter on the `template` module that runs a command against a temporary file before the real file is replaced. If the command returns non-zero, the original file is left untouched and the task fails.

```
- name: Deploy the nginx configuration safely
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
    validate: 'nginx -t -c %s'
  notify: Reload nginx

- name: Deploy the sshd configuration safely
  ansible.builtin.template:
    src: sshd_config.j2
    dest: /etc/ssh/sshd_config
    owner: root
    group: root
    mode: '0600'
    validate: '/usr/sbin/sshd -t -f %s'
  notify: Restart sshd
```

In the `validate` command, `%s` is replaced with the path to the temporary file. The command must exit 0 for the file to be moved into place.

4. Handlers and Templates Together

Handlers and templates work best as a pair. The template module is idempotent: it writes the destination file only when the rendered content differs from what is already there. When it does write, it reports a change, which triggers the handler. When the content is already correct, no change is reported, the handler is not notified, and the service keeps running.

The canonical pattern:

```
---
- name: Deploy and run the application
  hosts: appservers
  become: true

  vars:
    app_port: 8080
    app_workers: 4
    log_level: info

  tasks:
    - name: Deploy the application configuration
      ansible.builtin.template:
        src: app.conf.j2
        dest: /etc/myapp/app.conf
        owner: deploy
        group: deploy
        mode: '0640'
        backup: true
        validate: '/opt/myapp/bin/myapp --check-config %s'
      notify:
        - Reload application
        - Clear application cache

  handlers:
    - name: Reload application
      ansible.builtin.service:
        name: myapp
        state: reloaded

    - name: Clear application cache
      ansible.builtin.command: /opt/myapp/bin/clear-cache
      changed_when: false
```

Observe that two handlers are notified by a single task. Both run at the end of the play, in declaration order: the service is reloaded first, then the cache is cleared. If the configuration file is unchanged between runs, neither handler runs and the application is left untouched.

5. Practical Examples

5.1 nginx Virtual Host

Variable definition (group_vars/webservers.yml):

```
nginx_sites:
- name: example.com
  port: 80
  root: /var/www/example.com
  ssl: false
- name: api.example.com
  port: 443
  root: /var/www/api
  ssl: true
  certificate: /etc/ssl/certs/api.crt
  key: /etc/ssl/private/api.key
```

Template (templates/site.conf.j2):

```
{% for site in nginx_sites %}
server {
    listen {{ site.port }}{% if site.ssl %} ssl{% endif %};
    server_name {{ site.name }};

    root {{ site.root }};
    index index.html index.htm;

{% if site.ssl %}
    ssl_certificate      {{ site.certificate }};
    ssl_certificate_key  {{ site.key }};
    ssl_protocols        TLSv1.2 TLSv1.3;
{% endif %}

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/{{ site.name }}.access.log;
    error_log /var/log/nginx/{{ site.name }}.error.log;
}

{% endfor %}
```

Task (site.yml):

```
- name: Deploy nginx virtual hosts
  ansible.builtin.template:
    src: site.conf.j2
    dest: /etc/nginx/conf.d/sites.conf
    owner: root
    group: root
    mode: '0644'
    validate: 'nginx -t -c %s'
  notify: Reload nginx
```

5.2 Application Configuration from Facts

Facts make templates naturally host-specific. The following template generates a monitoring configuration that describes the host it runs on.

```
# templates/host-info.j2
hostname:    {{ ansible_facts.hostname }}
fqdn:       {{ ansible_facts.fqdn }}
os:         {{ ansible_facts.distribution }} {{ ansible_facts.distribution_version }}
kernel:     {{ ansible_facts.kernel }}
architecture: {{ ansible_facts.architecture }}
processor_count: {{ ansible_facts.processor_vcpus }}
memory_mb:  {{ ansible_facts.memtotal_mb }}

interfaces:
{% for iface, details in ansible_facts.interfaces | dictsort %}
{% if iface != 'lo' %}
  - name: {{ iface }}
    ipv4: {{ ansible_facts[iface].ipv4.address | default('none') }}
{% endif %}
{% endfor %}

mounts:
{% for mount in ansible_facts.mounts %}
{% if mount.fstype not in ['tmpfs', 'devtmpfs', 'squashfs'] %}
  - device: {{ mount.device }}
    path:    {{ mount.mount }}
    size:    {{ mount.size_total | human_readable }}
{% endif %}
{% endfor %}
```

The template uses `dictsort` to iterate over interfaces in a deterministic order, the `default` filter to guard against interfaces without an IPv4 address, and the `human_readable` filter to format byte counts as KB / MB / GB.

5.3 Multi-Host Configuration File

A common pattern is to generate a single configuration file on every host that lists all peers in the group. The `groups` variable makes this easy.

```
# templates/cluster.conf.j2
# Cluster membership for {{ inventory_hostname }}
# Generated from the {{ group_names | join(', ') }} groups

[cluster]
name = {{ cluster_name | default('default') }}

[members]
{% for host in groups['appservers'] | sort %}
{% if host != inventory_hostname %}
{{ host }} = {{ hostvars[host].ansible_facts.default_ipv4.address }}
{% endif %}
{% endfor %}

[seeds]
{% for host in groups['appservers'] | sort %}
seed.{{ loop.index }} = {{ hostvars[host].ansible_facts.default_ipv4.address }}
{% endfor %}
```

This template uses `groups['appservers']` to iterate over peer hosts, `hostvars` to retrieve facts from each peer, and `sort` to produce a stable ordering. The result is identical on every host, which is what cluster membership files require.

6. Best Practices

- **Use handlers for every service restart.** Never put `state: restarted` directly in a task. Notify a handler instead.
- **Match notify to handler names exactly.** Typos cause silent failures that are hard to debug.
- **Flush handlers when a later task depends on the restart.** Use `meta: flush_handlers`.
- **Chain handlers when order matters.** Have the first handler notify the second to enforce a strict sequence.
- **Use listen topics to decouple.** Notify by topic rather than by handler name when several events should trigger the same action.
- **Add a header comment to every template.** State that the file is generated and should not be edited by hand.
- **Always validate configuration files.** Pass `validate` to the `template` module when the target application offers a syntax checker.
- **Use the default filter liberally.** `{{ value | default('fallback') }}` makes templates robust to missing variables.
- **Keep templates readable.** Break long conditionals across lines, comment complex sections, and avoid deeply nested loops.
- **Store templates next to the tasks that use them.** Within a role, keep them under `templates/`; for a flat playbook, use a project-level `templates/` directory.
- **Prefer template over copy for anything variable.** If the file content changes between hosts or environments, it belongs in a template.
- **Test with `--check --diff`.** The diff output shows exactly what a template will change before the file is written.

7. Quick Reference

Handlers

Goal	Syntax
Notify a handler	<code>notify: Restart nginx</code>
Notify by topic	<code>notify: reload application</code>
Define a handler	<code>handlers:\n - name: Restart nginx\n service: ...</code>
Listen for multiple topics	<code>listen:\n - reload application\n - restart application</code>
Flush handlers mid-play	<code>meta: flush_handlers</code>
Force a notification	<code>changed_when: true\nnotify: Reload application</code>
Force handlers on failure	<code>ansible-playbook site.yml --force-handlers</code>

Table 2 — Handlers quick reference.

Templates (Jinja2)

Goal	Syntax
Inject a variable	<code>{{ app_port }}</code>
Provide a default	<code>{{ app_port default(80) }}</code>
Conditional block	<code>{% if ssl %}...{% else %}...{% endif %}</code>
Iterate over a list	<code>{% for s in servers %}...{% endfor %}</code>
Iterate over a dictionary	<code>{% for k, v in cfg dictsort %}...{% endfor %}</code>
Loop metadata	<code>loop.index, loop.first, loop.last</code>
Trim whitespace	<code>{%- ... -%}</code>
Include another template	<code>{% include "snippet.j2" %}</code>
Template inheritance	<code>{% extends "base.j2" %}{% block x %}...{% endblock %}</code>
Join a list	<code>{{ servers join(', ') }}</code>
Serialize to JSON	<code>{{ config to_json }}</code>
Comment in a template	<code>{# this is a Jinja2 comment #}</code>
Escape Jinja2 syntax	<code>{{ "{{" }}</code> for a literal brace
Deploy a template	<code>template:\n src: x.j2\n dest: /etc/x</code>
Validate before writing	<code>validate: 'nginx -t -c %s'</code>

Table 3 — Jinja2 templates quick reference.

This document describes Ansible Core behaviour as of version 2.14 and later and Jinja2 as bundled with that release. Template syntax and handler behaviour may evolve between releases; always consult the official documentation for the version in use.